

PROCESAMIENTO DE DATOS MASIVOS

Actividad 2 – Programación en Spark

Ejercicio 2

Enunciado

Dado un dataset que contiene información sobre los videos de Youtube (<https://netsg.cs.sfu.ca/youtubedata/>), **crear un programa llamado `CategoriaDeVideosMasVista` que obtenga cuál es la categoría de videos más vista de la plataforma Youtube y el número total de visualizaciones que hay en esa categoría.** El programa debe recibir dos parámetros de entrada: la carpeta en la que está el dataset y la carpeta en la que se guardará el resultado. En la carpeta donde está el dataset se tienen que descomprimir UNO de los archivos 0222.zip, 0301.zip, etc., que se encuentran en el enlace anterior. Importante: si la persona que hace la actividad dispone de pocos recursos computacionales, entonces se recomienda que únicamente descomprima algún .zip pequeño para que pueda desarrollar el programa. La carpeta de datos de entrada debería quedar como se ve en la Figura. Los datos de entrada están en los archivos 0.txt, 1.txt, etc y cada fila contiene la información de un video tabulada con el siguiente formato: id del video de youtube, usuario que subió el video, número de días desde que se subió el video y la fecha en la que obtuvieron los datos, categoría del video, longitud del video, número de visitas del video, puntuación del video, número de puntuaciones del video, número de comentarios del video, y una lista de ids de videos relacionados. Se valorará positivamente la eficiencia del programa, por ejemplo no usar transformaciones innecesarias.

Solución

Se propone la solución contenida en la carpeta “ej2”, compuesta por los siguientes documentos:

- Datos de entrada: carpeta 0301 con varios .txt: 0.txt, 1.txt...
- Código: se han desarrollado el script `categoriaDeVideosMasVista.py`
- Comando de ejecución: el código a escribir en la terminal se reporta en `terminal_categoriaDeVideoMasVista.txt`
- Resultado: el resultado generado por el programa se reporta en la carpeta `categoriaDeVideoMasVista`

Explicación Código Script

```
1  from pyspark import SparkContext
2
3  sc = SparkContext(appName="CategoriaDeVideosMasVista")
4
5  input_folder = "/user/dani/0302/[0-9]*.txt"
6  output_folder = "/user/dani/actividad213"
7
8  rdd = sc.textFile(input_folder)
9
10 def process_line(line):
11     columns = line.split("\t")
12     if len(columns) > 6:
13         category = columns[3].strip()
14         try:
15             views = float(columns[5])
16         except ValueError:
17             views = 0.0
18         return (category, views)
19     else:
20         return None
21
22 def filter_none_and_empty(x):
23     return x is not None and x[0] != ""
24
25 def sum_views(x, y):
26     return x + y
27
28 def sort_by_views(item):
29     # Para orden descendente, devolvemos el negativo
30     return -item[1]
31
32 def take_first_partition(index, iterator):
33     if index == 0:
34         for i, row in enumerate(iterator):
35             if i == 0:
36                 yield f"{row[0]};{int(row[1])}"
37     # las demás particiones no devuelven nada
38
39 processed_rdd = rdd.map(process_line).filter(filter_none_and_empty)
40
41 category_views_rdd = processed_rdd.reduceByKey(sum_views)
42
43 # Ordenamos por visitas descendente
44 ordered_rdd = category_views_rdd.sortBy(sort_by_views)
45
46 # Tomamos solo la primera fila en el RDD
47 top_rdd = ordered_rdd.mapPartitionsWithIndex(take_first_partition)
48
49 # Guardamos el resultado
50 top_rdd.saveAsTextFile(output_folder)
51
52 sc.stop()
```

Este script en *PySpark* tiene como objetivo encontrar la categoría de vídeos con mayor número de visualizaciones a partir de ficheros de texto ubicados en HDFS. Para ello, se crea primero un contexto *Spark* con el objeto *sc*, lo cual permite procesar datos de forma distribuida. La ruta de entrada se define en la variable *input_folder* y corresponde a archivos con extensión *.txt* que se encuentran bajo la carpeta */user/dani/0302/*. Cuando se ejecuta *sc.textFile(input_folder)*, se obtiene el RDD llamado *rdd*, que contiene todas las líneas de texto de esos ficheros, donde cada elemento del RDD es una línea completa del archivo en forma de cadena de texto. Cada línea representa información de un vídeo, con distintos campos separados por tabuladores, como ID del vídeo, título, nombre del canal, categoría, duración, número de visualizaciones, entre otros.

Para procesar estas líneas, se define la función **process_line**. Esta función divide cada línea en una lista llamada *columns* mediante el método *split("\t")*. Comprueba si la línea tiene al menos siete columnas para evitar errores de índices. Si es así, extrae la categoría desde *columns[3]* y el número de visualizaciones desde *columns[5]*, intentando convertirlo a un número flotante; si ocurre un error en la conversión, se asigna el valor 0.0 para evitar fallos. La función devuelve una tupla (*category, views*) o bien *None* si la línea no es válida. Después, se crea el RDD **processed_rdd** aplicando *map(process_line)* sobre *rdd*, lo que transforma cada línea en una tupla o en *None*. Sobre este RDD, se utiliza *.filter(filter_none_and_empty)*, que es una función que descarta todos los elementos *None* o aquellos cuya categoría esté vacía, para asegurarse de que sólo se mantengan datos válidos. Como resultado, **processed_rdd** contiene exclusivamente pares (categoría, número de visualizaciones) para vídeos válidos.

Luego, se agrupan las visualizaciones por categoría mediante *reduceByKey(sum_views)*, que se aplica sobre **processed_rdd** y genera un nuevo RDD llamado **category_views_rdd**. Aquí, la función *sum_views* simplemente suma los valores de visualizaciones de cada categoría. El resultado de **category_views_rdd** es un conjunto de tuplas donde cada elemento representa una categoría y la suma total de visualizaciones acumuladas de todos los vídeos que pertenecen a ella.

Para saber cuál es la categoría más vista, el script necesita ordenar los datos. Esto se logra sobre **category_views_rdd** usando el método *.sortBy(sort_by_views)*, que devuelve el RDD **ordered_rdd**. La función *sort_by_views* devuelve el valor negativo de las visualizaciones, asegurando un orden descendente, de modo que las categorías con más visualizaciones queden al principio del RDD. No obstante, Spark reparte los datos en múltiples particiones, y el primer elemento global puede estar en cualquiera de ellas, así que no basta con tomar el primer elemento de *ordered_rdd*. Por eso, se define la función *take_first_partition*, la cual se aplica mediante *mapPartitionsWithIndex* sobre *ordered_rdd* para crear el RDD *top_rdd*. Esta función procesa únicamente la primera partición (índice 0) y, dentro de ella, selecciona sólo el primer registro, formateándolo en una cadena de texto del tipo *"categoria;numero_de_visualizaciones"*.

Finalmente, *top_rdd* se guarda en HDFS en la ruta indicada por la variable *output_folder* usando *saveAsTextFile(output_folder)*, y se cierra el contexto *Spark* con *sc.stop()*. En resumen, el script toma datos en crudo en *rdd*, los procesa en **processed_rdd**, agrupa y suma las visualizaciones en **category_views_rdd**, ordena los resultados en **ordered_rdd** y finalmente extrae y guarda la categoría más vista en **top_rdd**.

Ejemplo:

```
a123 Video One ChannelA Music 5:00 1000 2023-01-01
b234 Video Two ChannelB Sports 4:30 2500 2023-01-02
c345 Video Three ChannelC Music 3:45 1500 2023-01-03
d456 Video Four ChannelD Travel 10:00 3000 2023-01-04
```

- **rdd = sc.textFile(input_folder)**

```
[
    "a123\tVideo One\tChannelA\tMusic\t5:00\t1000\t2023-01-01",
    "b234\tVideo Two\tChannelB\tSports\t4:30\t2500\t2023-01-02",
    "c345\tVideo Three\tChannelC\tMusic\t3:45\t1500\t2023-01-03",
    "d456\tVideo Four\tChannelD\tTravel\t10:00\t3000\t2023-01-04"
]
```

- **processed_rdd = rdd.map(process_line).filter(filter_none_and_empty)**

```
[
    ("Music", 1000.0),
    ("Sports", 2500.0),
    ("Music", 1500.0),
    ("Travel", 3000.0)
]
```

- **category_views_rdd = processed_rdd.reduceByKey(sum_views)**

```
[
    ("Music", 2500.0), # 1000 + 1500
    ("Sports", 2500.0),
    ("Travel", 3000.0)
]
```

- **ordered_rdd = category_views_rdd.sortBy(sort_by_views)**

```
[  
  ("Travel", 3000.0),  
  ("Music", 2500.0),  
  ("Sports", 2500.0)  
]
```

- **top_rdd = ordered_rdd.mapPartitionsWithIndex(take_first_partition)**

```
[  
  "Travel;3000"  
]
```

- **Resultado final**

```
Travel;3000
```

Ejecución

Seguimos los siguientes pasos para ejecutar el programa:

1. Ubicarnos en la ruta deseada (cd /home/dani/)
2. Dar permiso de ejecución a los scripts (chmod u+x categoriaDeVideoMasVista.py)
3. Activamos el yarn: start-yarn.sh y el DFS: start-dfs.sh
4. Ejecutar el programa:

```
dani@dani-VMware-Virtual-Platform:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
dani@dani-VMware-Virtual-Platform:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [dani-VMware-Virtual-Platform]
```

```
dani@dani-VMware-Virtual-Platform:~$ spark-submit --master yarn categoriaDeVideoMasVista.py
25/07/10 00:39:49 WARN Utils: Your hostname, dani-VMware-Virtual-Platform resolves to a loopback address: 127.0.1.1; using 192.168.176.131 instead (on interface ens33)
25/07/10 00:39:49 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
25/07/10 00:39:50 INFO SparkContext: Running Spark version 3.4.4
25/07/10 00:39:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
25/07/10 00:39:50 INFO ResourceUtils: =====
=====
25/07/10 00:39:50 INFO ResourceUtils: No custom resources configured for spark.driver.
25/07/10 00:39:50 INFO ResourceUtils: =====
=====
25/07/10 00:39:50 INFO SparkContext: Submitted application: CategoriaDeVideosMasVista
25/07/10 00:39:50 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
25/07/10 00:39:50 INFO ResourceProfile: Limiting resource is cpus at 1 tasks per executor
```

Pasamos los datos obtenidos del Hadoop File System a local:

```
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -get /user/dani/categoriaDeVideoMasVista /home/dani/categoriaDeVideoMasVista
dani@dani-VMware-Virtual-Platform:~$
```

Resultado

```
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -cat /user/dani/categoriaDeVideoMasVista/part-00000  
Music;26922840
```

Music;26922840